PRACT 2 EMail

October 31, 2026

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

[5]:
```
# Load dataset
df = pd.read_csv(r"E:\LP3_LAB Manual\ML PRAC\PRACT2\email\emails.csv")

# Display first five rows
df.head()
```

[5]:

	Email No.	the	to	ect	and	for	of	a	you	hou	...	connevey	jay	\
0	Email 1	0	0	1	0	0	0	2	0	0	...	0	0	
1	Email 2	8	13	24	6	6	2	102	1	27	...	0	0	
2	Email 3	0	0	1	0	0	0	8	0	0	...	0	0	
3	Email 4	0	5	22	0	5	1	51	2	10	...	0	0	
4	Email 5	7	6	17	1	5	2	57	0	9	...	0	0	

	valued	lay	infrastructure	military	allowing	ff	dry	Prediction
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	0	0
2	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0

```
4       0    0                0          0          0   1    0           0

[5 rows x 3002 columns]
```

[6]:
```
# Display dataset information
print(df.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5172 entries, 0 to 5171
Columns: 3002 entries, Email No. to Prediction
dtypes: int64(3001), str(1)
memory usage: 118.5 MB
None
```

[7]:
```
# Check for missing values
print(df.isnull().sum())
```

```
Email No.     0
the           0
to            0
ect           0
and           0
             ..
military      0
allowing      0
ff            0
dry           0
Prediction    0
Length: 3002, dtype: int64
```

[8]:
```
# Check class distribution
print(df["Prediction"].value_counts())
```

```
Prediction
0    3672
1    1500
Name: count, dtype: int64
```

[12]:
```
# Remove missing values
df = df.dropna()
df=df.drop(["Email No."],axis=1)
# Separate input features and target
X = df.drop("Prediction", axis=1)
Y = df["Prediction"]

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    Y,
```

```
    test_size=0.25,
    random_state=42,
    stratify=Y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (3879, 3000)
Testing data: (1293, 3000)
```

[13]:
```
# Create KNN model with feature scaling
knn = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", KNeighborsClassifier(n_neighbors=5))
])

# Train the model
knn.fit(X_train, y_train)

# Predict test data
knn_pred = knn.predict(X_test)

# Evaluate model
print("KNN Classification Report")
print(classification_report(
    y_test,
    knn_pred,
    target_names=["Not Spam", "Spam"],
    zero_division=0
))

# Calculate performance metrics
knn_accuracy = accuracy_score(y_test, knn_pred)
knn_precision = precision_score(y_test, knn_pred, zero_division=0)
knn_recall = recall_score(y_test, knn_pred, zero_division=0)
knn_f1 = f1_score(y_test, knn_pred, zero_division=0)

print("KNN Accuracy:", knn_accuracy)
print("KNN Precision:", knn_precision)
print("KNN Recall:", knn_recall)
print("KNN F1 Score:", knn_f1)

# Confusion matrix
knn_cm = confusion_matrix(y_test, knn_pred)

sns.heatmap(
```

```
    knn_cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Not Spam", "Spam"],
    yticklabels=["Not Spam", "Spam"]
)

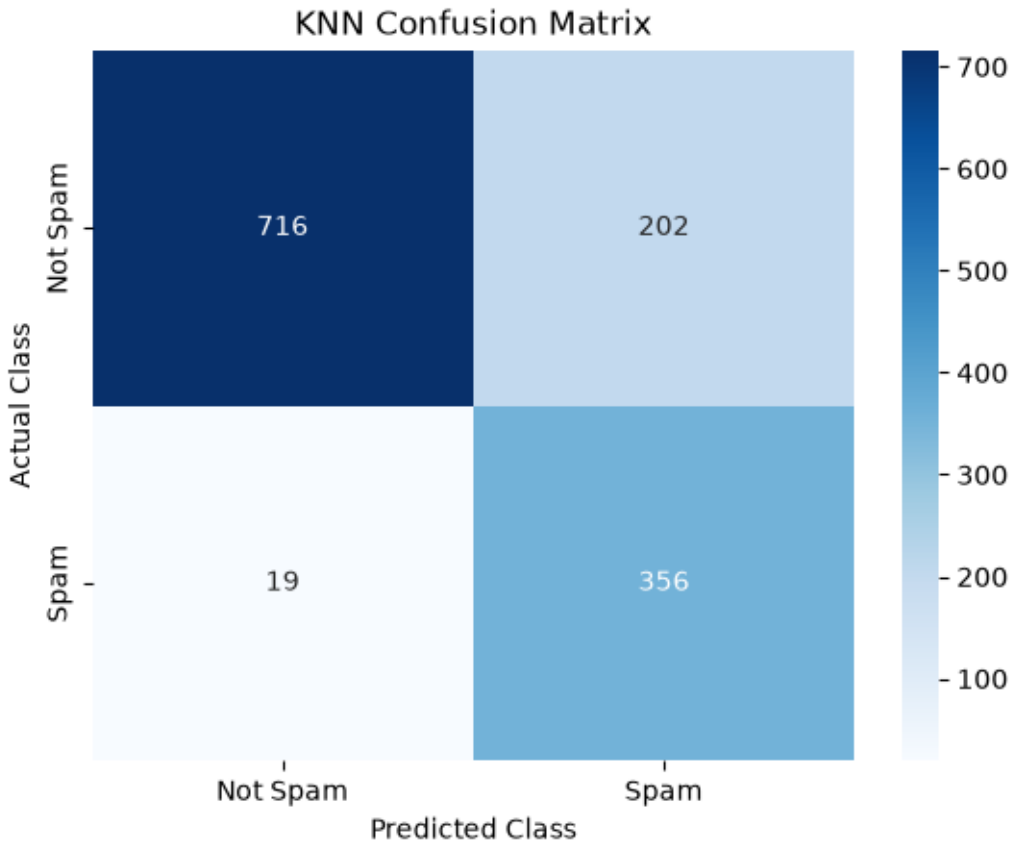
plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
```

```
KNN Classification Report
              precision    recall  f1-score   support

    Not Spam       0.97      0.78      0.87       918
        Spam       0.64      0.95      0.76       375

    accuracy                           0.83      1293
   macro avg       0.81      0.86      0.81      1293
weighted avg       0.88      0.83      0.84      1293

KNN Accuracy: 0.8290796597061099
KNN Precision: 0.6379928315412187
KNN Recall: 0.9493333333333334
KNN F1 Score: 0.7631296891747053
```



[14]:
```python
# Create SVM model with feature scaling
svm = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", SVC(kernel="linear"))
])

# Train the model
svm.fit(X_train, y_train)

# Predict test data
svm_pred = svm.predict(X_test)

# Evaluate model
print("SVM Classification Report")
print(classification_report(
    y_test,
    svm_pred,
    target_names=["Not Spam", "Spam"],
    zero_division=0
))
```

```python
# Calculate performance metrics
svm_accuracy = accuracy_score(y_test, svm_pred)
svm_precision = precision_score(y_test, svm_pred, zero_division=0)
svm_recall = recall_score(y_test, svm_pred, zero_division=0)
svm_f1 = f1_score(y_test, svm_pred, zero_division=0)

print("SVM Accuracy:", svm_accuracy)
print("SVM Precision:", svm_precision)
print("SVM Recall:", svm_recall)
print("SVM F1 Score:", svm_f1)

# Confusion matrix
svm_cm = confusion_matrix(y_test, svm_pred)

sns.heatmap(
    svm_cm,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=["Not Spam", "Spam"],
    yticklabels=["Not Spam", "Spam"]
)

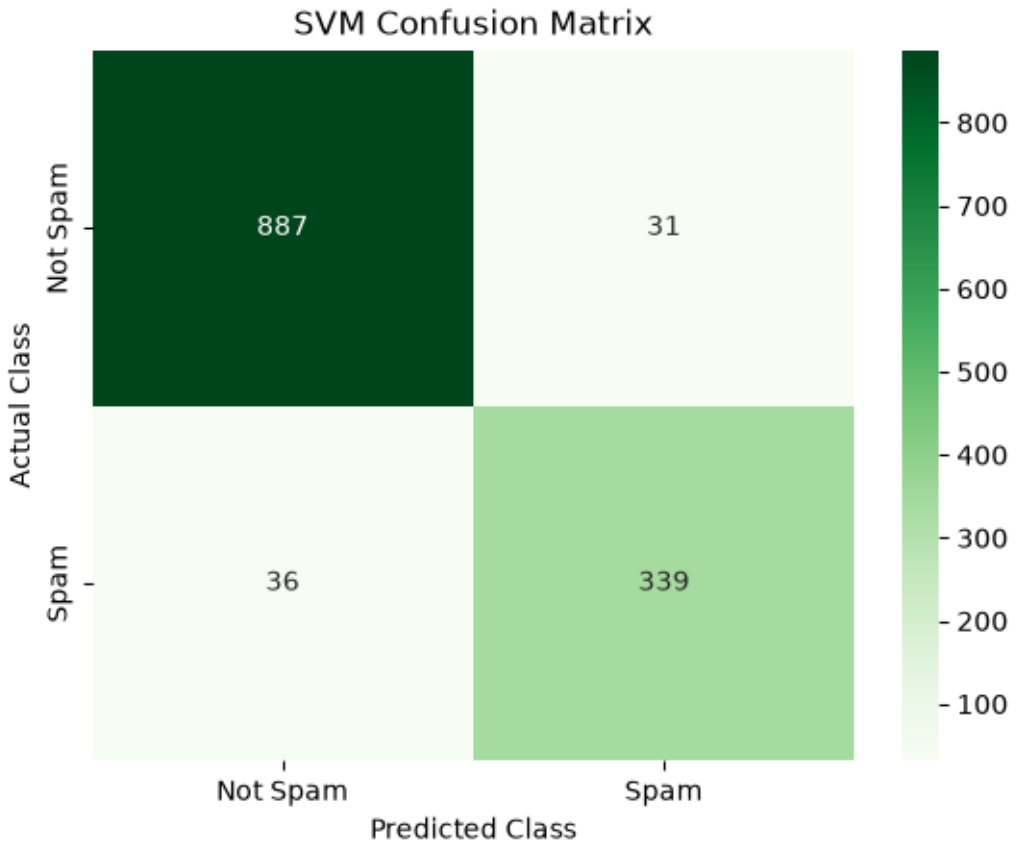
plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
```

```
SVM Classification Report
              precision    recall  f1-score   support

    Not Spam       0.96      0.97      0.96       918
        Spam       0.92      0.90      0.91       375

    accuracy                           0.95      1293
   macro avg       0.94      0.94      0.94      1293
weighted avg       0.95      0.95      0.95      1293

SVM Accuracy: 0.9481825212683681
SVM Precision: 0.9162162162162162
SVM Recall: 0.904
SVM F1 Score: 0.9100671140939597
```



[15]:
```python
# Create comparison table
comparison = pd.DataFrame({
    "Algorithm": ["KNN", "SVM"],
    "Accuracy": [knn_accuracy, svm_accuracy],
    "Precision": [knn_precision, svm_precision],
    "Recall": [knn_recall, svm_recall],
    "F1 Score": [knn_f1, svm_f1]
})

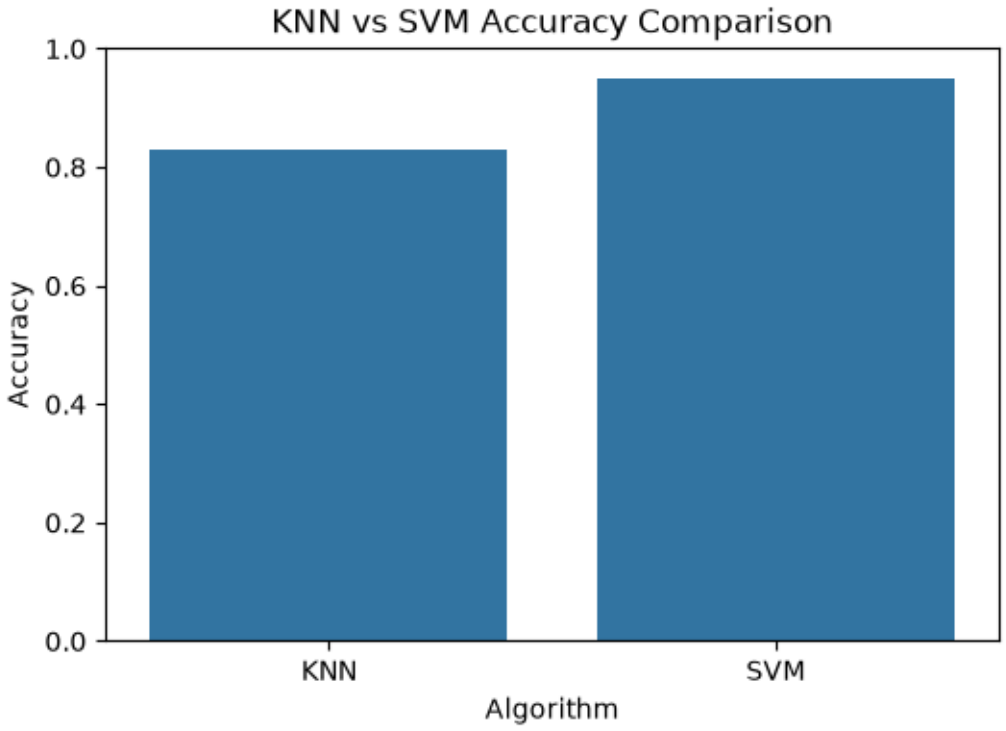
print(comparison)

# Plot accuracy comparison
plt.figure(figsize=(6, 4))

sns.barplot(
    data=comparison,
    x="Algorithm",
    y="Accuracy"
)
```

```
plt.title("KNN vs SVM Accuracy Comparison")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.show()
```

```
  Algorithm  Accuracy  Precision    Recall  F1 Score
0       KNN  0.829080   0.637993  0.949333  0.763130
1       SVM  0.948183   0.916216  0.904000  0.910067
```



[16]:
```
# Enter feature values for a new email
new_email = pd.DataFrame(
    [[0] * X.shape[1]],
    columns=X.columns
)

# Predict using KNN
knn_result = knn.predict(new_email)

# Predict using SVM
svm_result = svm.predict(new_email)

# Display results
if knn_result[0] == 1:
```

```
    print("KNN: Spam Email")
else:
    print("KNN: Not Spam Email")

if svm_result[0] == 1:
    print("SVM: Spam Email")
else:
    print("SVM: Not Spam Email")
```

```
KNN: Spam Email
SVM: Spam Email
```

[ ]: